

Pacific National Bank operates a synthetic yet high-fidelity core banking estate designed to emulate enterprise operating patterns used by large regulated financial institutions. The environment is intentionally broad, spanning legacy transaction processing on COBOL and PL/I, IMS hierarchical data access paths, and a modern Java service tier that supports branch, ATM, mobile, and online channels. The primary objective of this dataset is to provide a realistic software topology for architecture exercises, migration planning, and operational rehearsal without exposing customer data or production secrets.

At the center of the estate is the account and customer domain. Customer records model legal identity, household relationships, regulatory status, and lifecycle indicators such as onboarding stage and KYC completion. Account records represent demand deposits, savings, term instruments, and lending products, each with product controls, pricing metadata, and posting constraints. Transaction events are recorded with both accounting and channel semantics so that operations teams can reconcile branch-level movement, network settlement, and ledger-level journal impact.

Nightly processing is represented by batch orchestration definitions. End-of-day sequences include posting finalization, suspense clearing, statement composition, regulatory extracts, and integrity checks. Recovery-oriented JCL examples include restart points and dependency ordering to illustrate practical operations patterns in mixed criticality windows. In addition, synthetic reconciliation files and balancing controls demonstrate how operational accounting teams would verify cash positions and control totals before opening the next business day.

The online processing stream models teller-assisted cash operations and self-service transfers. Input messages carry transaction intent, account identifiers, amount values, and override indicators. Validation layers enforce basic product limits, status checks, and account ownership rules. Error paths intentionally include standardized return blocks and trace identifiers to mimic service desk diagnostics and audit follow-up flows.

A generator framework in Python supports deterministic artifact expansion from YAML models. It translates domain metadata into copybooks, include members, Java classes, SQL DDL, and documentation stubs. The framework is intended to be maintainable rather than opaque: models remain human-readable, templates stay modular, and output is reproducible for pipeline execution.

Database assets include DB2 definitions for canonical entities and migration scripts that model incremental schema evolution. IMS-aligned metadata captures segment relationships required for transaction routing and program specification binding. The Java tier packages core rules, channel APIs, and integration adapters that map external partner interactions into internal contracts.

Finally, this repository emphasizes operational traceability. Scripts and workflows demonstrate staged checks, targeted tests, and packaging controls suitable for enterprise release governance. While synthetic, the codebase is intentionally coherent enough to support design reviews, automation exercises, and training for teams working across hybrid mainframe and cloud-native environments.

Operationally, service ownership is split across product-aligned teams with shared controls for resilience engineering and incident response. Each release cycle includes synthetic